

`addContact` and `deleteContact` methods, which will dispatch the add and delete actions we had discussed earlier.

We need to access the Redux store to receive the state changes and to dispatch actions. The React container component `ContactsComponent` will interface with the store using the connect API provided by the `react-redux` module. The connect API accepts two parameters – a function (which we will name `mapStateToProps`) that receives the state from the store, and another function (which we will name `mapDispatchToProps`) that receives the dispatch method. The return value of the connect API is a function to which we will pass the React container component `ContactsComponent` as a parameter, and then finally export it.

```
class ContactsComponent extends React.Component {
  render() {
    // Return the JSX to display Contact list and the
    Contact form
  }
}

function mapStateToProps(state, ownProps) {
  // Return an object containing state details
}

function mapDispatchToProps(dispatch) {
  // Return an object containing action dispatch details
}

export default connect( mapStateToProps, mapDispatchToProps )(ContactsComponent);
```

Create the file `ContactForm.js` under the folder `components`. We will implement the code to get the form inputs from the user for adding a contact. This will be a React presentation component as it need not directly interface with the store. It will pass the input information to the container component, `ContactsComponent`, which takes care of interfacing with the store.

```
export default class ContactForm extends React.Component {
  render() {
    // Return the JSX to display input elements
  }
}
```

Create the file `ContactComponent.js` under the folder `components`. We will implement the code to render the contact information:

```
export default class ContactComponent extends React.Component {
  render() {
    // Return the JSX for a contact
  }
}
```

```
}
}
```

Implementing the startup code

In the file `index.js` under the `src` folder, remove the code generated by `create-react-app` and add the code to implement the startup code of the application. We will invoke the `configureStore` function discussed earlier. Next, we will dispatch the action to load the contacts. Then we will render the top level `ContactsComponent` by wrapping it inside a `Provider` component provided by the `react-redux` module. We wrap it inside a `Provider` component to provide access to the Redux store through the connect API of `react-redux`, which we had discussed earlier.

```
store = configureStore();

store.dispatch(loadContacts());

// Render the Contacts React component by wrapping it in
Provider
```

Updating `index.html` to include stylesheet

In the file `index.html` under the `public` folder, add the link to the bootstrap CSS for presentable UI and the stylesheet `toastr.css` to display UI status during add and delete actions.

```
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/
libs/twitter-bootstrap/3.3.7/css/bootstrap.min.css">
<link rel="stylesheet" href="css/toastr.css"/>
```

Remove the generated files that are not needed

Remove the files `App.js`, `App.css`, `App.test.js`, `index.css` and `logo.svg` generated by `create-react-app` as we do not need them.

Running the application

1. Go to the Node.js command prompt. Change the directory to `contacts`.
2. Start the server.

```
start node server.js
```

3. Start the React application Contacts:

```
npm start
```

This will start the application and display the Contacts information. We can now add and delete contacts. 

By: Srinivasan M.

The author works at Wipro and has experience in various JavaScript frameworks. He can be reached at srinivasan@yahoo.co.in